

Assignment-11.3

2303A51242

Batch-05

Task-1: Smart Contact Manager (Arrays & Linked Lists)Scenario

Prompt: "Generate a Python Smart Contact Manager using both a list (array) and a singly linked list. Each contact should store name and phone number. Implement add_contact(), search_contact(), and delete_contact() methods in both approaches. For the linked list, create a Node class and properly handle deletion of head and middle nodes. Use clear comments and simple logic. Finally, compare the array-based and linked list implementations in terms of insertion efficiency, deletion efficiency, memory usage, and time complexity in table format."

Code:

```
class ContactManager:
    def __init__(self):
        self.contacts = []
    def add_contact(self, name, phone_number):
        """Add a new contact to the contact manager."""
        contact = {'name': name, 'phone_number': phone_number}
        self.contacts.append(contact)
        print(f"Contact '{name}' added successfully.")
    def search_contact(self, name):
        """Search for a contact by name and return the phone number."""
        for contact in self.contacts:
            if contact['name'] == name:
                print(f"Contact found: {contact['name']} - {contact['phone_number']}")
                return contact['phone_number']
        print(f"Contact '{name}' not found.")
        return None
    def delete_contact(self, name):
        """Delete a contact by name."""
        for contact in self.contacts:
            if contact['name'] == name:
                self.contacts.remove(contact)
                print(f"Contact '{name}' deleted successfully.")
                return
        print(f"Contact '{name}' not found. Cannot delete.")

# Example usage
if __name__ == "__main__":
    manager = ContactManager()
    manager.add_contact("Alice", "123-456-7890")
    manager.add_contact("Bob", "987-654-3210")

    manager.search_contact("Alice")
    manager.search_contact("Charlie")

    manager.delete_contact("Bob")
    manager.delete_contact("Charlie")

class Node:
    def __init__(self, name, phone_number):
```

```

class Node:
    def __init__(self, name, phone_number):
        self.name = name
        self.phone_number = phone_number
        self.next = None

class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone_number):
        """Add a new contact to the linked list."""
        new_node = Node(name, phone_number)
        if self.head is None:
            self.head = new_node
            print(f"Contact '{name}' added successfully.")
            return

        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
        print(f"Contact '{name}' added successfully.")

    def search_contact(self, name):
        """Search for a contact by name and return the phone number."""
        current_node = self.head
        while current_node:
            if current_node.name == name:
                print(f"Contact found: {current_node.name} - {current_node.phone_number}")
                return current_node.phone_number
            current_node = current_node.next
        print(f"Contact '{name}' not found.")
        return None

    def delete_contact(self, name):
        """Delete a contact by name."""
        current_node = self.head
        previous_node = None
        while current_node:
            if current_node.name == name:
                if previous_node is None: # Deleting head node

```

```

class ContactManagerLinkedList:
    def search_contact(self, name):
        return current_node.phone_number
        current_node = current_node.next
    print(f"Contact '{name}' not found.")
    return None

    def delete_contact(self, name):
        """Delete a contact by name."""
        current_node = self.head
        previous_node = None
        while current_node:
            if current_node.name == name:
                if previous_node is None: # Deleting head node
                    self.head = current_node.next
                else: # Deleting middle or last node
                    previous_node.next = current_node.next
                print(f"Contact '{name}' deleted successfully.")
                return
            previous_node = current_node
            current_node = current_node.next
        print(f"Contact '{name}' not found. Cannot delete.")

# Example usage
if __name__ == "__main__":
    manager = ContactManagerLinkedList()
    manager.add_contact("Alice", "123-456-7890")
    manager.add_contact("Bob", "987-654-3210")
    manager.search_contact("Alice")
    manager.search_contact("Charlie")
    manager.delete_contact("Bob")
    manager.delete_contact("Charlie")

```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python3
ASSISTANT CODING/assignment-11.py"
Contact 'Alice' added successfully.
Contact 'Bob' added successfully.
Contact found: Alice - 123-456-7890
Contact 'Charlie' not found.
Contact 'Bob' deleted successfully.
Contact 'Charlie' not found. Cannot delete.
Contact 'Alice' added successfully.
Contact 'Bob' added successfully.
Contact found: Alice - 123-456-7890
Contact 'Charlie' not found.
Contact 'Charlie' not found.
Contact 'Bob' deleted successfully.
Contact 'Charlie' not found. Cannot delete.
Contact 'Alice' added successfully.
Contact 'Bob' added successfully.
Contact found: Alice - 123-456-7890
Contact 'Charlie' not found.
Contact 'Charlie' not found. Cannot delete.
Contact 'Alice' added successfully.
Contact 'Bob' added successfully.
Contact found: Alice - 123-456-7890
Contact 'Charlie' not found.
Contact 'Bob' added successfully.
Contact found: Alice - 123-456-7890
Contact 'Charlie' not found.
Contact found: Alice - 123-456-7890
Contact 'Charlie' not found.
Contact 'Bob' deleted successfully.
Contact 'Charlie' not found. Cannot delete.
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The code defines two classes, `Contact Manager` and `Contact Manager LinkedList`, to manage contacts using a list and a linked list, respectively. Each class has methods

#to add, search, and delete contacts. The `Contact Manager` class uses a list to store contacts, while the `Contact Manager LinkedList` class uses a linked list structure. Both classes handle cases where a contact is not found gracefully by printing appropriate messages instead of crashing the program. The example usage demonstrates how to use both classes to manage contacts effectively.

Task 2: Library Book Search System (Queues & Priority Queues)Scenario

Prompt: "Generate a Python Priority Queue implementation using the `heapq` module where faculty requests have higher priority than student requests. Implement `enqueue(name, role)` and `dequeue()` methods. Include `clear`

comments and demonstrate the system with sample test cases showing that faculty requests are served before student requests."

Code:

```
from collections import deque
class Queue:
    def __init__(self):
        self.queue = deque()
    def enqueue(self, item):
        self.queue.append(item)
        print(f"Enqueued: {item}")
    def dequeue(self):
        if not self.is_empty():
            item = self.queue.popleft()
            print(f"Dequeued: {item}")
            return item
        else:
            print("Queue is empty. Cannot dequeue.")
            return None
    def is_empty(self):
        return len(self.queue) == 0
# Simulating book request processing in FIFO order
if __name__ == "__main__":
    book_queue = Queue()
    # Enqueue book requests
    book_queue.enqueue("Book 1")
    book_queue.enqueue("Book 2")
    book_queue.enqueue("Book 3")
    # Dequeue book requests
    book_queue.dequeue()
    book_queue.dequeue()
    book_queue.dequeue()
    book_queue.dequeue() # Attempt to dequeue from an empty queue
import heapq
class PriorityQueue:
    def __init__(self):
        self.queue = []
        self.counter = 0 # To maintain the order of requests with the same priority
    def enqueue(self, name, role):
        priority = 0 if role == "faculty" else 1 # Faculty has higher priority
        heapq.heappush(self.queue, (priority, self.counter, name))
```

```

        self.counter += 1
        print(f"Enqueued: {name} ({role})")
    def dequeue(self):
        if not self.is_empty():
            priority, count, name = heapq.heappop(self.queue)
            print(f"Dequeued: {name}")
            return name
        else:
            print("Priority Queue is empty. Cannot dequeue.")
            return None
    def is_empty(self):
        return len(self.queue) == 0
# Demonstrating with sample inputs
if __name__ == "__main__":
    priority_queue = PriorityQueue()
    # Enqueue requests
    priority_queue.enqueue("Alice", "student")
    priority_queue.enqueue("Bob", "faculty")
    priority_queue.enqueue("Charlie", "student")
    priority_queue.enqueue("David", "faculty")
    # Dequeue requests
    priority_queue.dequeue() # Should dequeue Bob (faculty)
    priority_queue.dequeue() # Should dequeue David (faculty)
    priority_queue.dequeue() # Should dequeue Alice (student)
    priority_queue.dequeue() # Should dequeue Charlie (student)
    priority_queue.dequeue() # Attempt to dequeue from an empty queue
    "Provide a test case demonstrating faculty requests being served before student requests."
if __name__ == "__main__":
    priority_queue = PriorityQueue()
    # Enqueue requests
    priority_queue.enqueue("Alice", "student")
    priority_queue.enqueue("Bob", "faculty")
    priority_queue.enqueue("Charlie", "student")
    priority_queue.enqueue("David", "faculty")
    # Dequeue requests

```

```

        return len(self.queue) == 0
# Demonstrating with sample inputs
if __name__ == "__main__":
    priority_queue = PriorityQueue()
    # Enqueue requests
    priority_queue.enqueue("Alice", "student")
    priority_queue.enqueue("Bob", "faculty")
    priority_queue.enqueue("Charlie", "student")
    priority_queue.enqueue("David", "faculty")
    # Dequeue requests
    priority_queue.dequeue() # Should dequeue Bob (faculty)
    priority_queue.dequeue() # Should dequeue David (faculty)
    priority_queue.dequeue() # Should dequeue Alice (student)
    priority_queue.dequeue() # Should dequeue Charlie (student)
    priority_queue.dequeue() # Attempt to dequeue from an empty queue
    "Provide a test case demonstrating faculty requests being served before student requests."
if __name__ == "__main__":
    priority_queue = PriorityQueue()
    # Enqueue requests
    priority_queue.enqueue("Alice", "student")
    priority_queue.enqueue("Bob", "faculty")
    priority_queue.enqueue("Charlie", "student")
    priority_queue.enqueue("David", "faculty")
    # Dequeue requests
    priority_queue.dequeue() # Should dequeue Bob (faculty)
    priority_queue.dequeue() # Should dequeue David (faculty)
    priority_queue.dequeue() # Should dequeue Alice (student)
    priority_queue.dequeue() # Should dequeue Charlie (student)
    priority_queue.dequeue() # Attempt to dequeue from an empty queue

```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING\assignment-11.py
ASSISTANT CODING/assignment-11.py"
Enqueued: Book 1
Enqueued: Book 2
Enqueued: Book 3
Dequeued: Book 1
Dequeued: Book 2
Dequeued: Book 3
Queue is empty. Cannot dequeue.
Enqueued: Alice (student)
Enqueued: Bob (faculty)
Enqueued: Charlie (student)
Enqueued: David (faculty)
Dequeued: Bob
Dequeued: David
Dequeued: Alice
Dequeued: Charlie
Priority Queue is empty. Cannot dequeue.
Enqueued: Alice (student)
Enqueued: Bob (faculty)
Enqueued: Charlie (student)
Enqueued: David (faculty)
Dequeued: Bob
Dequeued: David
Dequeued: Alice
Dequeued: Charlie
Priority Queue is empty. Cannot dequeue.
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

```
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

```
Priority Queue is empty. Cannot dequeue.
PS C:\Users\kamil\OneDrive\Desktop\AI ASSISTANT CODING>
```

```
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
Priority Queue is empty. Cannot dequeue.  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

```
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>  
Priority Queue is empty. Cannot dequeue.  
Priority Queue is empty. Cannot dequeue.  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

```
Priority Queue is empty. Cannot dequeue.  
Priority Queue is empty. Cannot dequeue.  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The Queue class implements a simple FIFO queue using deque, where items are added at the end and removed from the front. The PriorityQueue class uses a min-heap (heapq) to assign higher priority to faculty (0) over students (1). The enqueue method adds requests with priority, and dequeue removes the highest-priority request first. The test cases show that faculty requests are served before student requests and that both classes handle empty queue conditions safely without errors.

Task-3: Task 3: Emergency Help Desk (Stack Implementation)Scenario

Prompt: "Generate a Python class implementing a Stack data structure using a list.

Include methods: push(), pop(), peek(), is_empty(), and is_full().

Simulate at least five tickets and demonstrate LIFO behavior."

"Suggest additional validation checks for stack overflow and underflow conditions in Python implementation." combine these two and give single prompt

Code:

```
#"Generate a Python class implementing a Stack data structure using a list.
#Include methods: push(), pop(), peek(), is_empty(), and is_full().
#Simulate at least five tickets and demonstrate LIFO behavior."
```

```
class Stack:
    def __init__(self, capacity):
        self.stack = []
        self.capacity = capacity

    def push(self, item):
        if len(self.stack) < self.capacity:
            self.stack.append(item)
            print(f"Ticket '{item}' added to the stack.")
        else:
            print("Stack is full. Cannot add more tickets.")

    def pop(self):
        if not self.is_empty():
            item = self.stack.pop()
            print(f"Ticket '{item}' removed from the stack.")
            return item
        else:
            print("Stack is empty. No tickets to remove.")
            return None

    def peek(self):
        if not self.is_empty():
            print(f"Top ticket in the stack: '{self.stack[-1]}')
            return self.stack[-1]
        else:
            print("Stack is empty. No tickets to peek.")
            return None

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
```

```
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) == self.capacity

# Simulating ticket operations
ticket_stack = Stack(capacity=5)
ticket_stack.push("Ticket 1")
ticket_stack.push("Ticket 2")
ticket_stack.push("Ticket 3")
ticket_stack.peek()
ticket_stack.push("Ticket 4")
ticket_stack.push("Ticket 5")
ticket_stack.push("Ticket 6") # This should indicate the stack is full
ticket_stack.pop()
ticket_stack.peek()

"Suggest additional validation checks for stack overflow and underflow conditions in Python implementation."
# In the Stack class implementation, we can add additional validation checks for stack overflow and underflow conditions.
# For stack overflow, we can check if the stack has reached its maximum capacity before allowing a new item to be pushed. If the st
# For stack underflow, we can check if the stack is empty before allowing an item to
#be popped. If the stack is empty, we can print a message indicating that there are no items to remove.
# These checks are already implemented in the push() and pop() methods of the Stack class, ensuring that the program handles these
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe ASSISTANT_CODING/assignment-11.py
Ticket 'Ticket 1' added to the stack.
Ticket 'Ticket 2' added to the stack.
Ticket 'Ticket 3' added to the stack.
Top ticket in the stack: 'Ticket 3'
Ticket 'Ticket 4' added to the stack.
Ticket 'Ticket 5' added to the stack.
Stack is full. Cannot add more tickets.
Ticket 'Ticket 5' removed from the stack.
Ticket 'Ticket 3' added to the stack.
Top ticket in the stack: 'Ticket 3'
Ticket 'Ticket 4' added to the stack.
Ticket 'Ticket 5' added to the stack.
Stack is full. Cannot add more tickets.
Ticket 'Ticket 5' removed from the stack.
Top ticket in the stack: 'Ticket 4'
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The Stack class includes methods to manage a stack data structure with a specified capacity. The push() method checks for stack overflow before adding an item, while the pop() method checks for stack underflow before removing an item. The peek() method allows us to see the top item without modifying the stack. The example usage demonstrates how to use the Stack class and shows the LIFO behavior of the stack.

Task-4: Hash Table

Prompt: "Generate a Python class for a Hash Table with collision handling using chaining.

Include methods: insert(key, value), search(key), delete(key).

Use a list of lists to implement chaining.

Add proper comments explaining the hash function and collision handling."

Code:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)] # Create a list of lists for chaining

    def hash_function(self, key):
        """Simple hash function that uses the built-in hash and modulo operator."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self.hash_function(key)
        # Check if the key already exists in the chain
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value) # Update existing key
                return
        # If key does not exist, add new key-value pair to the chain
        self.table[index].append((key, value))

    def search(self, key):
        """Search for a value by its key in the hash table."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v # Return the value if key is found
        return None # Return None if key is not found

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
```

```
            index = self.hash_function(key)
            for k, v in self.table[index]:
                if k == key:
                    return v # Return the value if key is found
            return None # Return None if key is not found

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i] # Remove the key-value pair from the chain
                return True # Return True if deletion was successful
        return False # Return False if key was not found

# Example usage
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name")) # Output: Alice
print(hash_table.search("age")) # Output: 30
hash_table.delete("name")
print(hash_table.search("name")) # Output: None
# Explanation: The `HashTable` class implements a hash table with collision handling using chaining. The `hash_function` method com
```

Output:

```
ASSISTANT CODING/assignment-11.py"  
Alice  
30  
None  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The `HashTable` class implements a hash table with collision handling using chaining. The `hash\_function` method computes the index for a given key using Python's built-in `hash` function and the modulo operator. The `insert` method adds key-value pairs to the appropriate index, handling collisions by appending to a list at that index. The `search` method looks for a key in the chain and returns its value if found, while the `delete` method removes a key-value pair from the chain if it exists.

Task-5: Real-Time Application Challenge Scenario

Prompt: # "Suggest appropriate data structures for the following system features:

Student Attendance Tracking

Event Registration System

Library Book Borrowing

Bus Scheduling System

Cafeteria Order Queue

Justify each choice in 2–3 sentences."

#1. Student Attendance Tracking: A suitable data structure for this feature would be a hash table (or dictionary) where the key is the student ID and the value is a list of attendance records (dates and times).

This allows for efficient lookups and updates when marking attendance, as well as easy retrieval of a student's attendance history.

#2. Event Registration System: A linked list could be an appropriate data structure for managing event registrations.

Each node in the linked list can represent a registration, containing information such as the registrant's name, contact details, and event details.

This allows for dynamic memory allocation and easy insertion or deletion of registrations without needing to shift elements, which is beneficial for handling varying numbers of registrations.

#3. Library Book Borrowing: A queue data structure would be suitable for managing the borrowing of books in a library. When a book is borrowed, it can be added to the end of the queue, and when a book is returned, it can be removed from the front of the queue. This ensures that books are borrowed and returned in a first-come, first-served manner, which is fair for all patrons.

#4. Bus Scheduling System: A priority queue would be an appropriate data structure for managing bus schedules. Each bus can be assigned a priority based on factors such as departure time, route, or passenger demand. This allows the system to efficiently manage and schedule buses based on their priority, ensuring that buses with higher priority are scheduled first.

#5. Cafeteria Order Queue: A queue data structure would be ideal for managing the cafeteria order queue. Customers can place their orders, which are added to the end of the queue, and the cafeteria staff can process orders in the order they were received. This ensures a fair and organized system for handling customer orders, preventing confusion and ensuring timely service.

Feature	Data Structure	Justification
Student Attendance	Hash Table	Fast lookup using roll number as key
Event Registration	Linked List	Dynamic addition/removal
Library Borrowing	Queue	FIFO system
Bus Scheduling	Graph	Stops and routes mapping
Cafeteria Orders	Queue	First come, first serve

Prompt: #"Generate a Python implementation for a Cafeteria Order Queue system using Queue data structure.

#Include methods for placing and serving orders.

#Demonstrate with sample test cases."

Code:

```
#Generate a Python implementation for a Cafeteria Order Queue system using Queue data structure.
#Include methods for placing and serving orders.
#Demonstrate with sample test cases."
from collections import deque
class CafeteriaOrderQueue:
    def __init__(self):
        self.order_queue = deque()

    def place_order(self, order):
        """Place an order in the queue."""
        self.order_queue.append(order)
        print(f"Order placed: {order}")

    def serve_order(self):
        """Serve the next order in the queue."""
        if self.order_queue:
            order = self.order_queue.popleft()
            print(f"Order served: {order}")
            return order
        else:
            print("No orders to serve.")
            return None

# Example usage
cafeteria = CafeteriaOrderQueue()
cafeteria.place_order("Burger")
cafeteria.place_order("Fries")
cafeteria.place_order("Soda")
cafeteria.serve_order() # Should serve "Burger"
cafeteria.serve_order() # Should serve "Fries"
cafeteria.serve_order() # Should serve "Soda"
cafeteria.serve_order() # Should indicate no orders to serve
```

Output:

```
ASSISTANT CODING/assignment-11.py"
Order placed: Burger
Order placed: Fries
Order placed: Soda
Order served: Burger
Order served: Fries
Order served: Soda
None
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The `CafeteriaOrderQueue` class uses a `deque` from the `collections` module to implement the queue data structure. The `place\_order` method adds an order to the end of the queue, while the `serve\_order` method removes and returns the order at the front of the queue. If there are no orders to serve, it prints a message indicating that the queue is empty. The example usage demonstrates placing several orders and serving them in the correct order, as well as handling the case when there are no more orders to serve.

